

New Project Handoff

For the agent starting a new, standalone project in the repository `mayerspitz/test`

Project details come from the user in chat. This document covers only how to work: the isolation rules, documentation, the user's preferences, and the conventions. The user will give you the project's name, goal, requirements and first tasks directly. Where this document says `<project-name>`, use the name the user gives you.

0. What this is

You're starting a **new project** in an existing repository that is only its temporary home. Other, unrelated projects live on other branches of this repo (for example a home audio system on `claude/upbeat-edison-2k76xq`). **Ignore them and never touch them.** This project will one day become **its own repository**, so build it as if it already were one.

1. Isolation rules (non-negotiable)

1. Branch:

- Work only on the branch your session assigns you.
- Create it from `origin/main`, never from another `claude/*` branch: `git fetch origin main && git checkout -b <your-branch> origin/main`.
- **Never merge into main**, and open no pull request unless the user asks.
- Push only your own branch: `git push -u origin <your-branch>`.

2. One top-level folder:

everything lives in `<project-name>/`. Files go at the repo root only if a host strictly requires it (e.g. a `render.yaml` blueprint). Keep those minimal and pointing into your folder.

3. Nothing shared:

- No imports from, symlinks to, or copies of code in other folders or branches.
- Your folder has its own dependency manifest, lockfile, tests, docs and `.gitignore`.

4. Deployment:

- Create your **own** hosting services, named after the project.
- **Never modify, redeploy or delete** services of other projects (on Render: leave `home-audio` and `home-audio-demo` alone).

5. Secrets:

- Never commit passwords, tokens or keys. Put them in the host's environment variables.
- `.env.example` lists names only.
- If the user types a secret in chat, use it only in env settings, and redact it in any log you write.

6. **Easy to extract later:** relative paths only, with no assumptions about the parent repo. Moving the folder to its own repo should need no code changes.

2. Folder layout to start with

```
<project-name>/
  README.md           what it is, how to run and test it, live URLs
  CLAUDE.md           agent rules for this project (copy sections 1, 3 and 4 here)
  docs/
    PROJECT_CONTEXT.md single source of truth (see section 3)
    OPEN_ITEMS.md      live to-do list and questions for the user, with status
    history/
      CONVERSATION_LOG.md every user message word for word (secrets redacted)
                          plus a one-line summary of each reply
      RESEARCH_NOTES.md  verified facts, with source links
  src/ ... test/ ...   whatever the stack needs
```

3. Documentation discipline: the user wants nothing lost

- **PROJECT_CONTEXT.md** is read first by every agent and every resumed or compacted session. It holds:
 - the user's working rules
 - the user's environment and constraints
 - the requirements
 - a **numbered decision log** (date, decision, *why*, alternatives rejected)
 - what's built, and where it runs (URLs, service IDs, env var *names*)
 - what's verified and what isn't yet
 - next steps
- **When a decision, answer or fact changes**, update `PROJECT_CONTEXT.md` and `OPEN_ITEMS.md` in the same commit. Add each new user message to `history/CONVERSATION_LOG.md`.
- Every open question goes in `OPEN_ITEMS.md` with a status (waiting / answered, and where the answer is recorded). The user follows up there later.
- **Documents from other agents:** if the user hands you one, don't trust it blindly.
 - List where it contradicts what the user has said since.
 - Confirm those points with the user before building on them or recording them as decisions.

4. How this user likes to work

- **Don't ask what you can work out, or what they already told you.**
 - Derive answers from facts already given (e.g. compute distances from known dimensions).

- Ask only questions that truly change the outcome.
- Batch and number them (P1, P2, ...), and say what each answer would change.
- **As automatic as possible:**
 - One-line installs, and self-checks that explain problems in plain English.
 - No manual dashboard steps for the user. If a host's API can't do something, find a workaround in code.
- **Quality over convenience:** off-the-shelf solutions are a plus, never at the cost of the end result.
- **Finish, test, then report:**
 - Run the full test suite before every push.
 - Try UIs in a real browser at phone size (Playwright with Chromium is preinstalled).
 - Confirm deploys are live.
- **Be honest about limits:** say plainly what was verified and what wasn't (e.g. real hardware, or sites the sandbox can't reach).
- **How to reply:**
 - Plain English. Lead with the outcome. Use short tables and lists.
 - Before any purchase, give exact products, quantities, approximate prices and totals.
- The user uses **Android** but prefers **web-based** apps (mobile-first, installable to the home screen) unless told otherwise.
- **Mid-task messages:** the user often adds instructions while you're working. Fold them in without dropping the current work, and confirm in your reply that they were handled.

5. Repository and tooling conventions

- **Commits:** clear messages, ending with the attribution lines your session provides. No model names in commits, code or docs.
- **Deploys:**
 - Render has worked well for this user, and the free plan is fine unless they say otherwise. Record in the docs that free services sleep and lose their disk on restart.
 - Auto-deploy from your branch, with the start command inside your folder (e.g. `cd <project-name> && npm start`).
 - Check each deploy's status after pushing.
- **Tests:** one command (e.g. `npm test`) runs everything. Keep the test count current in the README.
- **Sandbox network:** outbound HTTPS goes through a proxy, and some sites are blocked.
 - Prefer primary sources (source code on GitHub, official docs).
 - Cite them in `RESEARCH_NOTES.md`, and flag anything you couldn't confirm.

6. Moving to its own repository later

Only when the user says so:

1. Create the new, empty repo (this needs the user's permission and access).
2. Split out the folder with its history: `git subtree split --prefix=<project-name> -b <project-name>-export`.
3. Push that branch to the new repo as `main`. Move any root-level host files (e.g. `render.yaml`) into the new root and fix their paths.
4. Point the hosting service at the new repo and branch. Confirm it deploys, then update the URLs in `PROJECT_CONTEXT.md`.
5. Ask the user before deleting anything from the old repo.

7. First steps

1. Read the user's project details in chat.
2. Create your branch from `origin/main`, and scaffold the folder from section 2.
3. Fill in `PROJECT_CONTEXT.md` from the user's details, and log their first message.
4. Record open questions in `OPEN_ITEMS.md`. Ask the user only the ones that block you.
5. Build, test, commit, push, deploy, and report back.